

Тема 5. Контроллеры регистрации и входа

Задание: реализовать логику создания пользователя и проверки учётных данных.

Пошаговое руководство:

1. В `register.js` принять `req.body`, выполнить проверку на пустоту полей. Найти существующего пользователя через `Users.findOne({ where: { login } })`. Если найден, вернуть 400. Создать запись через `Users.create()`.
2. Установить куку `res.cookie('userId', newUser.id, { httpOnly: true, maxAge: 18 * 24 * 60 * 60 * 1000 })`. Вернуть 201.
3. В `login.js` принять `login` и `password`. Добавить проверку на демо-админа. Найти пользователя в БД. Сравнить пароли. Установить куку аналогично регистрации. Вернуть 200.
4. Обернуть всю логику в `try/catch`, при ошибке возвращать 500.

Контрольные вопросы:

1. Зачем в контроллерах используется конструкция `try...catch` и какой HTTP-статус должен возвращаться в блоке `catch`?
2. В чём разница между методами `Users.findOne()` и `Users.create()`? Что возвращает каждый из них и как они взаимодействуют с базой данных?
3. Почему прямое сравнение паролей через `!==` считается небезопасным для реальных проектов и какой подход рекомендуется использовать вместо этого?
4. Что произойдёт, если при регистрации отправить POST-запрос с логином, который уже существует в таблице? Какой статус и сообщение вернёт сервер?
5. Как правильно задать время жизни `cookie` через параметр `maxAge` и почему важно указывать его в миллисекундах?

Тема 6. Клиентская логика и финальное тестирование

Задание: связать фронтенд с бэкендом, настроить обработку форм и проверить полный цикл.

Пошаговое руководство:

1. В `public/js/index.js` получить форму и блок сообщений. Добавить обработчик `submit`. Отменить стандартное поведение через `e.preventDefault()`.
2. Считать значения полей через `form.querySelector('input[name="..."]').value`. Выполнить клиентскую валидацию (длина логина, пароля, телефона).
3. Отправить данные через `fetch('/api/auth/register', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(userData) })`.
4. Обработать ответ: если `res.ok`, показать сообщение, очистить форму, через `setTimeout` перенаправить на `user.html`. Иначе показать ошибку.

5. Аналогично настроить `login.js` с `fetch('/api/auth/login')`.
6. В `server.js` подключить контроллеры и добавить маршруты:
`app.post('/api/auth/register', register)` и `app.post('/api/auth/login', login)`.

Контрольные вопросы:

1. Зачем в обработчике отправки формы вызывается `e.preventDefault()`?
2. Почему перед отправкой данных через `fetch` объект `userData` необходимо преобразовать с помощью `JSON.stringify()`, а в заголовках явно указать `Content-Type: application/json`?
3. Как работает свойство `res.ok` в ответе `fetch` и чем оно удобнее прямой проверки `res.status === 200`?
4. Почему клиентская валидация не может заменить серверную проверку данных? Приведите пример, как пользователь может обойти клиентские ограничения.
5. Для чего в коде используется `setTimeout` перед редиректом на другую страницу и как это влияет на восприятие интерфейса пользователем?